

MODULE 08 · DOMAIN 2 - DATA STORE MANAGEMENT

Amazon S3 as a data lake

Storage classes, lifecycle, layout, file formats, retention and the optimisations that decide the bill.

DOMAIN 2 of 4 EXAM WEIGHT 26% MODULE 8 of 27

S3 questions are almost always about **cost over time**, not about storing the object.

The storage classes

CLASS	USE IT FOR
S3 Standard	Active data. No minimum duration, no retrieval fee
S3 Intelligent-Tiering	Unknown or changing access patterns. AWS moves it for you
S3 Standard-IA	Infrequent access, needs milliseconds. 30 day minimum
S3 One Zone-IA	Same, but reproducible data you could rebuild. One AZ only
S3 Glacier Instant Retrieval	Archive accessed perhaps quarterly. Milliseconds. 90 day minimum
S3 Glacier Flexible Retrieval	Archive. Minutes to hours. 90 day minimum
S3 Glacier Deep Archive	Compliance archive. 12 to 48 hours. 180 day minimum

ON THE EXAM

Cheaper storage always buys a longer commitment. That trade is what the questions test.

The numbers to memorise

- ◆ **Minimum durations.** Standard-IA and One Zone-IA 30 days. Both Glacier 90. Deep Archive 180.
- ◆ **Minimum billable object size** is 128 KB for the infrequent access and Glacier classes.
- ◆ **Glacier Instant Retrieval** returns in milliseconds. Flexible in minutes to hours.
- ◆ **Glacier Flexible** offers expedited in one to five minutes, bulk free in five to twelve hours.
- ◆ **Deep Archive** is twelve hours standard, forty eight bulk.

ON THE EXAM

Millions of small objects in an IA class are billed at 128 KB each, which can cost more than Standard.

S3 Express One Zone

- ◆ **Single-digit millisecond** first-byte latency. The fastest S3 class.
- ◆ **Directory buckets**, a different bucket type, in one Availability Zone.
- ◆ Up to ten times faster than S3 Standard on request-heavy workloads.
- ◆ **Higher storage price**, much lower request price. The reverse of the usual trade.
- ◆ **One AZ**, so it suits reproducible working data rather than the source of truth.

ON THE EXAM

It optimises requests, not storage. That makes it a fit for many small reads, not for an archive.

Request rates and prefixes

- ◆ **3,500 writes and 5,500 reads per second, per prefix.** Scales automatically.
- ◆ **More prefixes means more throughput,** because the limit is per prefix.
- ◆ **A 503 slow down response** means you are exceeding it, not that S3 is broken.
- ◆ **Partitioning already spreads keys** across prefixes, which helps here too.
- ◆ **Retry with backoff** is the correct client behaviour, as everywhere else.

ON THE EXAM

Partitioning by date happens to distribute keys across prefixes, so a well-organised lake rarely hits this.

Intelligent-Tiering or lifecycle rules?

INTELLIGENT-TIERING

AWS decides

- Monitors each object's access
- No retrieval fees, no minimum duration
- Small monitoring fee per object
- For unknown or changing patterns

LIFECYCLE RULES

You decide

- Transitions by age, on a fixed schedule
- No monitoring fee
- Wrong guess means retrieval charges
- For patterns you genuinely know

THE TIEBREAKER

"Unknown", "unpredictable" or "changing" access patterns is the Intelligent-Tiering signal, almost every time.

Lifecycle rules in practice

- ◆ **Transition actions** move objects to a cheaper class at a given age.
- ◆ **Expiration actions** delete objects at a given age.
- ◆ **Filter by prefix, tag or object size** so one bucket can have several policies.
- ◆ **Expire noncurrent versions** separately, or a versioned bucket grows forever.
- ◆ **Clean up incomplete multipart uploads.** They are billed and invisible.

ON THE EXAM

Incomplete multipart uploads are the classic invisible cost. They do not appear in the object listing and they are charged.

A company stores IoT telemetry in Amazon S3. Data is queried heavily for the first fourteen days, occasionally for the next three months, and then only if a regulator asks, which happens perhaps once a year. Regulation requires the data to be kept for seven years.

A data engineer proposes a lifecycle rule that transitions objects to S3 Glacier Deep Archive after fourteen days and expires them after seven years. Average object size is 4 megabytes.

What is wrong with the proposal?

- ☐ A Nothing. Deep Archive after fourteen days is the cheapest option available
- ☐ B Retrieval from Deep Archive takes up to forty eight hours, which does not meet the occasional access needed in months one to three
- ☐ C Objects smaller than 128 kilobytes cannot be transitioned to Deep Archive
- ☐ D Lifecycle rules cannot expire objects after more than five years

◆ PAUSE AND DECIDE

What is wrong with the proposal?

- A

 Nothing is wrong
Cheapest per gigabyte, and the question describes occasional access for three months that Deep Archive cannot serve.
- C

 Small objects cannot transition
The 128 KB rule is a billing minimum, not a prohibition. Objects here are 4 MB anyway.
- B

 Deep Archive is too slow for months one to three
Three distinct access patterns need three tiers. Standard, then an IA or Glacier Instant tier, then Deep Archive for the tail.
- D

 Five year expiry limit
Invented. Lifecycle expiration accepts any number of days.

THE SIGNAL

Three described access patterns need three tiers. Jumping straight to the cheapest ignores the middle one.

How you arrange objects decides what every query costs, **forever**.

Partitioning that works

- ◆ **Partition on what you filter by**, which is usually date.
- ◆ **Hive style**, a folder named year equals 2026, is understood by every engine.
- ◆ **Over-partitioning** makes thousands of tiny folders and slows metadata lookup.
- ◆ **High cardinality keys** like customer ID make terrible partitions.
- ◆ **Partition projection in Athena** avoids listing partitions at all.

ON THE EXAM

Aim for partitions in the hundreds of megabytes to a few gigabytes. Both extremes hurt.

Indexing, in a lake

- ◆ **Parquet stores min and max per column chunk**, so engines skip blocks that cannot match.
- ◆ **Sort the data** on your filter column and those statistics become far more effective.
- ◆ **Bucketing** hashes rows into a fixed number of files, which speeds repeated joins.
- ◆ **Glue Data Catalog partition indexes** speed up partition lookup on large tables.
- ◆ Randomly ordered data defeats block skipping. Every block looks like it might match.

ON THE EXAM

Partitioning skips folders. Column statistics skip blocks inside a file. Both matter, at different scales.

File formats, and what each is for

FORMAT	CHARACTER
CSV	Row based, uncompressed, no schema. Universal and inefficient
JSON	Row based, self describing, verbose. Common for ingestion
Avro	Row based, binary, strong schema evolution. Good for writes
Parquet	Columnar, compressed. The default for analytics
ORC	Columnar, compressed. Similar to Parquet, common with Hive

ON THE EXAM

Avro for write-heavy and evolving schemas. Parquet for read-heavy analytics. That is the whole distinction.

Compression, and the thing people forget

- ◆ **Snappy**. Fast, moderate ratio, splittable within Parquet. The usual default.
- ◆ **Gzip**. Better ratio, slower, and **not splittable** on its own.
- ◆ **Bzip2 and LZO** are splittable. Zstandard balances ratio and speed well.
- ◆ **One large gzip file** is read by exactly one task, however many workers you have.
- ◆ **Inside Parquet** compression is per block, so splittability is handled for you.

ON THE EXAM

Splittability decides parallelism. A non-splittable file is a single-threaded bottleneck no cluster can fix.

The layout that most lakes end up with

- ◆ **Raw.** Exactly as it arrived, never modified. The thing you can rebuild from.
- ◆ **Cleaned.** Validated, deduplicated, typed, converted to Parquet.
- ◆ **Curated.** Joined, aggregated, modelled for consumption.
- ◆ **Different lifecycle rules per layer.** Raw goes cold; curated stays warm.
- ◆ **Different permissions per layer.** Most people never need raw.

ON THE EXAM

Module six said the recovery story for derived data is the raw layer plus the code. This is where the raw layer lives.

Object tags, and what they are for

- ◆ **Up to ten tags per object**, key and value pairs, changeable without rewriting.
- ◆ **Lifecycle rules can filter on tags**, so policy follows classification.
- ◆ **IAM policies can filter on tags**, which is attribute-based access control.
- ◆ **Cost allocation tags** attribute spend to a team or project.
- ◆ Tags are metadata about the object, not a substitute for partitioning.

ON THE EXAM

Tagging an object as containing personal data lets one lifecycle rule and one IAM policy act on it everywhere.

An analytics team queries a 40 terabyte dataset in Amazon S3 with Amazon Athena. The data is stored as gzip-compressed CSV files, partitioned by year and month, with about 600 files per partition averaging 60 megabytes each.

Queries typically filter on a single month and read four columns out of ninety. They currently scan far more data than expected and are slow. The team wants both cost and runtime improved.

Which change will have the largest effect?

- ☐ A Repartition the data by day instead of by month
- ☐ B Convert the files to Apache Parquet with Snappy compression
- ☐ C Increase the Athena workgroup's query result reuse window
- ☐ D Move the data to S3 Standard-IA to reduce storage cost

◆ PAUSE AND DECIDE

Which change will have the largest effect?

- A

 Repartition by day
Helps only queries that filter by day. These filter by month, and month partitions are already being pruned correctly.
- B

 Convert to Parquet with Snappy
Four columns of ninety means reading roughly a twenty second of the data, and Snappy is splittable where gzip is not.
- C

 Longer result reuse window
Helps identical repeated queries. It does nothing for the first run or for varied queries.
- D

 Move to Standard-IA
Reduces storage cost and adds a retrieval fee to a frequently queried dataset. Athena bills bytes scanned regardless.

THE SIGNAL

Four columns out of ninety is the planted ratio. Row format means every query reads all ninety.

Some data must be kept. Some must be destroyed. Sometimes the same data, at different times.

Keeping data that must not be deleted

- ◆ **Versioning** keeps every version, so an overwrite or delete is recoverable.
- ◆ **A delete on a versioned bucket** adds a delete marker. Nothing is removed.
- ◆ **MFA delete** requires a second factor to remove a version permanently.
- ◆ **S3 Object Lock, governance mode.** Deletable by users with a specific permission.
- ◆ **Object Lock, compliance mode.** Nobody can delete it. Not even the root user.

ON THE EXAM

Compliance mode is genuinely irreversible. That is the point of it, and it is why questions about regulators name it.

Vault Lock, and why it still appears

- ◆ **S3 Glacier Vault Lock** is the older write-once mechanism, on Glacier vaults.
- ◆ A vault lock policy, once committed, cannot be changed for the life of the vault.
- ◆ **Vaults are the original Glacier API**, separate from S3 buckets and storage classes.
- ◆ **For new work, use S3 Object Lock** on Glacier storage classes instead.
- ◆ Both are cited for regulations such as SEC 17a-4. Recognise either.

ON THE EXAM

Two Glaciers exist. The storage classes inside S3, and the older standalone vault service. Questions blur them.

Deleting data that must go

- ◆ **Lifecycle expiration** for deletion by age. Automatic and cheap.
- ◆ **Deleting one person's records** is harder, because they sit inside shared files.
- ◆ **Open table formats** make row-level deletes possible on lake data.
- ◆ **S3 Batch Operations** for deleting or tagging millions of objects at once.
- ◆ **Object Lock and a deletion request** genuinely conflict. Design so they do not overlap.

ON THE EXAM

Row-level deletion in a lake is the problem Apache Iceberg solves. Module nine covers it properly.

Expiring data in DynamoDB

- ◆ **Time to Live** is a per-item attribute holding an expiry timestamp.
- ◆ **DynamoDB deletes expired items automatically**, at no cost in write capacity.
- ◆ **Deletion is not immediate**. It typically happens within a few days of expiry.
- ◆ **Expired items can still be returned** by queries until they are actually removed.
- ◆ **TTL deletions appear in DynamoDB Streams**, so they can be archived on the way out.

ON THE EXAM

TTL is the DynamoDB equivalent of a lifecycle expiration rule, and the exam guide names them in the same skill.

Two ways to let data expire

S3 LIFECYCLE EXPIRATION

By age, in bulk

- Applies to a prefix or a tag
- Deletes on a schedule S3 runs
- No per-object configuration
- Precise enough for retention policy

DYNAMODB TTL

Per item

- Each item carries its own expiry
- Different items expire at different times
- Deletion is eventual, within days
- Free of write capacity charges

THE TIEBREAKER

Different stores, same idea. The exam asks which mechanism belongs to which service, not which is better.

Resilience and durability

- ◆ **Eleven nines of durability**, across at least three Availability Zones.
- ◆ **One Zone-IA is the exception.** One AZ, so an AZ loss loses the data.
- ◆ **Cross-Region Replication** copies objects to another Region asynchronously.
- ◆ **Same-Region Replication** for account separation or log aggregation.
- ◆ **Replication Time Control** gives a fifteen minute replication SLA.

THE TIEBREAKER

Durability is not the same as availability, and neither protects you from deleting the wrong thing. Versioning does.

What replication actually does

- ◆ **Asynchronous.** Objects appear in the destination shortly after being written.
- ◆ **Only new objects** replicate by default. Existing ones need Batch Replication.
- ◆ **Replication Time Control** commits to 99.99 percent within fifteen minutes.
- ◆ **Delete markers replicate optionally.** Version deletions never do.
- ◆ **Can change storage class and owner** on the way, which suits an archive copy.

ON THE EXAM

Turning replication on does nothing to the data already in the bucket. That catches people in practice and in exams.

Encryption at rest, in brief

OPTION	WHAT IT MEANS
SSE-S3	AWS manages the key entirely. On by default for all new objects
SSE-KMS	A KMS key you control, with an audit trail of every use
DSSE-KMS	Two independent layers of encryption. For strict regulatory regimes
Client-side	You encrypt before upload. AWS never sees the plaintext

ON THE EXAM

All new objects are encrypted by default with SSE-S3. The question is who controls the key, not whether it is encrypted.

AWS Backup, and where it fits

- ◆ **Centralised backup policy** across S3, RDS, DynamoDB, EFS and more.
- ◆ **One retention policy** applied across services, with reporting for auditors.
- ◆ **Backup Vault Lock** gives write-once protection to the backups themselves.
- ◆ **Cross-account and cross-Region** copies for isolation from a compromised account.
- ◆ Overlaps with versioning and replication. It adds central policy and audit.

ON THE EXAM

On the in-scope service list and easy to overlook. It is the answer when the requirement spans several data stores.

Three S3 features that quietly closed

- ◆ **S3 Select.** Closed to new customers on 25 July 2024. No new capabilities.
- ◆ **S3 Glacier Select.** Closed at the same time.
- ◆ **S3 Object Lambda.** Existing customers and select partners only, from 7 November 2025.
- ◆ AWS points you at **Amazon Athena** or client-side filtering instead.
- ◆ All three appear in older banks as correct answers. Recognise them, know their status.

ON THE EXAM

Availability warning: S3 Select appears five times in the bank analysed for this course, and Object Lambda four.

Seeing what is in a lake

- ◆ **S3 Storage Lens** gives usage and activity trends across accounts and buckets.
- ◆ **Storage Class Analysis** watches access patterns and suggests transitions.
- ◆ **S3 Inventory** produces a scheduled report of objects and their metadata.
- ◆ **Inventory plus Batch Operations** is how bulk work gets done at lake scale.
- ◆ Listing a bucket with millions of objects to find something is slow and expensive.

ON THE EXAM

S3 Inventory is a flat file of everything in a bucket, delivered on a schedule. Query it with Athena.

Doing something to millions of objects

- ◆ **S3 Batch Operations** takes a manifest and performs one action on every object.
- ◆ Copy, replace tags, replace ACLs, restore from Glacier, or invoke a Lambda function.
- ◆ **The manifest** is an S3 Inventory report, or a CSV you generate with Athena.
- ◆ **It reports per-object success and failure**, which a script would have to build.
- ◆ **Restoring from Glacier in bulk** is a Batch Operations job, not a console action.

ON THE EXAM

Inventory to find them, Athena to filter them, Batch Operations to act on them. That chain answers a whole class of question.

A financial services company must retain trade records in Amazon S3 for seven years in a form that no employee, including administrators, can alter or delete. Separately, the same bucket holds customer support transcripts that must be erased within thirty days of a customer's deletion request.

A data engineer proposes enabling S3 Object Lock in compliance mode on the bucket with a seven year retention period.

What is the problem, and what should be done?

- ☐ A Nothing. Compliance mode satisfies both requirements
- ☐ B Compliance mode would make the support transcripts undeletable. Separate the two data types into different buckets with different policies
- ☐ C Use governance mode instead, so administrators can delete transcripts when asked
- ☐ D Enable versioning instead of Object Lock, and delete old versions as required

What is the problem, and what should be done?

- A

 Nothing is wrong
Compliance mode is absolute. The transcripts would be locked for seven years against a thirty day deletion obligation.
- B

 Separate the data, separate the policies
The two obligations are contradictory on the same objects. Different buckets let each have the policy it needs.
- C

 Governance mode instead
Makes the transcripts deletable and also makes the trade records deletable by anyone with the override permission. That fails the stated requirement.
- D

 Versioning instead of Object Lock
Versioning protects against accidents. It does not prevent a determined administrator from removing versions.

THE SIGNAL

Two datasets with opposite obligations cannot share one bucket policy. The answer is separation, not compromise.

Moving data in and out efficiently

- ◆ **Multipart upload** splits a large object into parts uploaded in parallel.
 - ◆ Recommended above 100 MB, required above 5 GB. Failed parts retry individually.
- ◆ **Transfer Acceleration** routes uploads over the CloudFront edge network.
 - ◆ Useful for uploads from far away. No help within the same Region.
- ◆ **Byte-range fetches** read part of an object in parallel.

ON THE EXAM

Transfer Acceleration only helps when distance is the problem. It is a distractor for same-Region questions.

What S3 actually charges for

CHARGE	NOTES
Storage per GB	Varies by class. The number people think of as the S3 bill
Requests	PUT, GET, LIST. Millions of small objects means millions of requests
Retrieval per GB	Standard-IA, One Zone-IA and the Glacier classes only
Early deletion	Charged to the class minimum if you delete or move sooner
Monitoring	Intelligent-Tiering, per object above 128 KB
Data transfer out	To the internet or across Regions. Free inbound, free within a Region

ON THE EXAM

Requests and retrieval are what turn a cheap storage class into an expensive one for the wrong access pattern.

Symptoms and their causes

SYMPTOM	LIKELY CAUSE
Bill higher than the visible data explains	Old versions retained, or incomplete multipart uploads
Queries scan far more than they need	Row format, or no partition pruning
A job will not parallelise	Non-splittable compression, or one huge file
Slow before any data is read	Millions of small objects to list and open
Cheaper storage class made the bill rise	128 KB minimum billing on tiny objects, or early deletion charges
Restored data unavailable for hours	Glacier Flexible or Deep Archive retrieval time

ON THE EXAM

Two of these rows describe a cost optimisation that made things worse. Both come from ignoring a minimum.

A company's S3 bill has grown steadily for a year while the volume of data the team knows about has stayed roughly constant at 30 terabytes. Storage Lens reports about 90 terabytes of billed storage in the bucket.

Versioning was enabled two years ago after an accidental deletion. A nightly job overwrites a set of aggregate files. Some large uploads from a flaky on-premises connection have failed over the year.

Which combination explains the gap and fixes it?

- ☐ A Enable S3 Intelligent-Tiering so infrequently accessed data moves to cheaper tiers
- ☐ B Add lifecycle rules to expire noncurrent versions and to abort incomplete multipart uploads
- ☐ C Disable versioning to stop old versions accumulating
- ☐ D Enable Cross-Region Replication so data can be deleted from the primary bucket

◆ PAUSE AND DECIDE

Which combination explains the gap and fixes it?

A Intelligent-Tiering

Makes the invisible data cheaper without removing it. You would still store 90 TB, just at a lower rate.

B Expire noncurrent versions, abort incomplete multipart uploads

A nightly overwrite on a versioned bucket keeps every previous version, and failed uploads leave billed parts that never appear in a listing.

C Disable versioning

Stops new versions accumulating, does not remove the existing ones, and throws away the protection they added it for.

D Cross-Region Replication

Copies the problem to a second Region and doubles the bill.

THE SIGNAL

Thirty terabytes known, ninety terabytes billed. Both planted details, versioning and failed uploads, explain the gap.

Designing buckets and prefixes

- ◆ **Bucket names are globally unique** and cannot be changed afterwards.
- ◆ **Separate buckets** when policy differs. Retention, encryption, replication, ownership.
- ◆ **Separate prefixes** when only the data differs, not the policy.
- ◆ **A bucket per table** becomes unmanageable and hits account bucket limits.
- ◆ **Cross-account access** through a bucket policy, or an S3 Access Point per consumer.

ON THE EXAM

The rule is simple. If two datasets need different policies, they need different buckets.

Sharing a lake with other accounts

- ◆ **A bucket policy** can grant another account access to the whole bucket or a prefix.
- ◆ **S3 Access Points** give each consumer its own endpoint and its own policy.
- ◆ Simpler than one bucket policy growing to hundreds of lines over time.
- ◆ **ACLs are disabled by default** on new buckets. Ownership is enforced by the bucket owner.
- ◆ **Requester Pays** makes the consumer pay for requests and transfer.

ON THE EXAM

Access Points exist because a single bucket policy becomes unmanageable once many teams need different slices.

An application writes session records to an Amazon DynamoDB table. Each session should be retained for exactly ninety days after its last activity, then removed. Sessions have wildly different lifespans, so their expiry dates differ.

The team also needs expired sessions archived to Amazon S3 for annual reporting before they disappear. A proposal is to run a nightly Lambda function that scans the table and deletes expired items.

What should the data engineer implement instead?

- ☐ A Keep the nightly scan, but increase the table's provisioned read capacity so it completes faster
- ☐ B Enable DynamoDB TTL on an expiry attribute, and use DynamoDB Streams to archive deleted items to Amazon S3
- ☐ C Apply an S3 Lifecycle expiration rule of ninety days to the DynamoDB table
- ☐ D Export the table to Amazon S3 every ninety days and recreate it empty

◆ PAUSE AND DECIDE

What should the data engineer implement instead?

- A

 Faster nightly scan
A full table scan every night consumes read capacity proportional to the whole table, to find a small fraction of it.
- B

 TTL plus Streams to S3
Per-item expiry handles the varying lifespans, deletion costs no write capacity, and the stream record carries the item out to S3.
- C

 S3 Lifecycle on a DynamoDB table
Category error. Lifecycle rules act on S3 objects and have no relationship to DynamoDB items.
- D

 Export and recreate every ninety days
Deletes items that are not yet ninety days old, and takes the table offline. It ignores the per-item requirement entirely.

THE SIGNAL

"Expiry dates differ per item" is what rules out any age-based bulk rule and points at TTL.

Reading an S3 question quickly

- ◆ **How does access change over time?** That is the lifecycle answer.
- ◆ **Is the pattern known or unknown?** Unknown means Intelligent-Tiering.
- ◆ **Check the minimums.** Object size and retention period both bite.
- ◆ **Is it cost of storage or cost of querying?** Completely different answers.
- ◆ **Must it be kept, or must it go?** Object Lock and erasure requests conflict.

ON THE EXAM

Next: open table formats. Apache Iceberg, S3 Tables, and the row-level operations a plain lake cannot do.